

BCS2420

Lab 4: Web Exploitations

Irina Iarlykanova

February 2026

Purpose of These Labs

Welcome to Lab 4! This lab is based on Lecture 6, and today we are going to explore web applications. While there are specific vulnerabilities in each programming language that developers should be aware of, there are issues fundamental to the internet that can appear regardless of the chosen language or framework.

It is almost impossible to solve web-based challenges without using any tools (or at least very difficult). That's why in this lab you will be introduced to new tools that you might have never seen before. But do not worry, we will guide you through them :)

Lastly, make sure you go over Lecture 6 before starting the lab. There will be no challenges that you cannot solve yourself. **Good luck!**

Challenge 1: "Subscribe to Read More"

Difficulty: Easy

Narrative Context

You are scrolling through *The Captivating Capital*, a finance magazine for people who say "summering" as a verb. An article catches your eye:

CookieVault: Inside the \$10-Per-Click Game Where Billionaires Burn Money for Sport

You need to know more about this platform and its players, but to read the article, you need to pay a lot of money!

Goal: Get the access to the full article by bypassing the paywall.

Key Learning Goals

This challenge introduces the following concepts:

- The Document Object Model (DOM)
- Browser Developer Tools (DevTools)

Concepts from Lecture

When you request a webpage, the server sends a copy of the code to your browser, where it is rendered and displayed. Everything you see in your browser was sent to your machine first.

Websites are made of:

- **Front End (Client-Side)** — runs in your browser, renders the webpage, handles interactions.
- **Back End (Server-Side)** — runs on the server, processes requests, returns responses.

Websites are primarily created using HTML, CSS and JavaScript. A web page is a document that can either be displayed in the browser, or as HTML source code in a program. *The DOM is an object representation of that web page* — a programming interface that enables a scripting language, such as JavaScript, to change the document structure, style and content of the web page.

Developer Tools (DevTools) is built into every modern browser. It allows you to view and manipulate the DOM, modify CSS in real time, debug JavaScript, and execute commands in the console.

For more information on how to use DevTools visit:

https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Tools_and_setup/What_are_browser_developer_tools

Challenge 2: “The Internet Never Forgets”

Difficulty: Easy

Narrative Context

So now you now, CookieVault, the game where the ultra-rich click a cookie to prove they're ultra-rich. You want to see who is playing the game, but there is no sign up. No waitlist. No "contact us." Just a login form and a wall of exclusivity.

You were not invited, but you can find another way in. Maybe someone got sloppy?

Goal: Find credentials and login with them to retrieve the flag.

Key Learning Goals

This challenge expands upon Challenge 1:

- Hidden but accessible resources
- Git repository metadata recovery

Concepts from Lecture

When you visit a website, your browser requests files from a server. Some of these files are meant for you to see (HTML pages, images, stylesheets), while others are meant to

stay hidden. But “hidden” does not always mean “inaccessible.” If you know what to look for, you might find files the developer never intended to expose.

There are many such files, but here are the essential ones to know:

File/Directory	Purpose	Meaning
<code>robots.txt</code>	Tells search engines which paths not to index	Often reveals directories the admin wants hidden from Google, but not protected from direct access
<code>.git/</code>	Git repository metadata and history	Full source code and history recoverable
<code>.env</code>	Stores environment variables (database credentials, API keys, secrets)	Often contains sensitive data in plaintext
<code>.bak</code> , <code>.old</code> , <code>.zip</code>	Backup or editor temporary files	May contain older versions with hardcoded credentials or vulnerabilities

Needed Tools

`git-dumper`

- Check if it is installed: `git-dumper --help`
- If you see `command not found`, install using: `pip install git-dumper`
- For more information: <https://github.com/arthaud/git-dumper>

Challenge 3: “Feed The Machine”

Difficulty: Medium

Narrative Context

Now that you are inside the CookieVault, you discover that these high-class people have an unusual hobby: competitive cookie clicking. The game is simple: click the cookie, and whoever has the most clicks wins the prize. The current leader has 999,999,999 clicks. Your goal is to claim first place.

Goal: To get the flag, you need to score first on the leaderboard.

Key Learning Goals

This challenge introduces the following concepts:

- HTTP request and response structure
- Session cookies and their role in authentication
- Signed cookies and cryptographic verification

Concepts from Lecture

HTTP Basics

When you interact with a website, your browser and the server communicate using HTTP (Hypertext Transfer Protocol). Every interaction follows a simple pattern:

1. **Request:** Your browser asks the server for something (a page, an image, or to submit data)
2. **Response:** The server replies with the requested content or a status message

A basic HTTP request looks like this:

```
GET /game HTTP/1.1
Host: example.com
Cookie: player_data=abc123
```

And a response:

```
HTTP/1.1 200 OK
Content-Type: text/html

<html>...</html>
```

Cookies

Cookies are small files of information that a web server generates and sends to a web browser.

A *session cookie* helps a website track a user's session. This cookie stores information such as the user's input and tracks the movements of the user within the website.

There is also such a thing as a *signed session cookie*. A signed cookie is a normal HTTP cookie whose value is accompanied by a cryptographic signature created with a server-side secret key. It is used so the server can detect tampering and trust the data stored in the cookie (for things like session or auth info) without storing it all server-side.

Needed Tools

- Browser Developer Tools (F12)
- (*Optional*) If you feel like a hacker or want to try a new (very popular and widely used) tool:
 - Burp Suite
 - In case of trouble visit: <https://portswigger.net/burp/communitydownload>

Challenge 4: “No Injector Will Pass!”

Difficulty: Medium

Narrative Context

Congratulations on scoring first in the previous challenge! I think it was cool, but the platform’s admins think otherwise... Unfortunately, they deleted your user, banned your persona and wished to not see you again. It sounds like you need to get someone else’s account to show them that you are a better hacker than they are programmers.

Goal: To get the flag, you need to login as `elan_maks`.

Key Learning Goals

This challenge introduces the following concepts:

- Authentication bypass via malicious queries

Concepts from Lecture

Almost no platform operates without a database. You have seen many databases, including MySQL, SQLite, MariaDB, MongoDB and many more.

Unfortunately for everyone, databases used to be super insecure. One of the most common and dangerous issues was (and still is) SQL injection: an attack where an attacker slips malicious SQL code into a normal input field (like a login form) so the application runs their query instead of the one the developer intended. In practice, this can let an attacker read or modify sensitive data, bypass authentication, or even delete whole tables. Modern applications defend against SQL injection by never concatenating raw user input into queries, and instead using prepared statements, parameterized queries, and strict input validation.

Some common SQL queries to test for SQLi:

- Confirm SQLi:

```
' OR 1=1--
```

- Error-Based Column Count:

```
1' ORDER BY 1 --  
1' ORDER BY 2 --
```

- UNION SELECT Discovery:

```
' UNION SELECT NULL --  
' UNION SELECT NULL,NULL --
```

Challenge 5: “Equalizer Weapon”

Difficulty: Hard

Narrative Context

I cannot believe you could fool the admins and get the `elan_maks` account — well done! It sounds like it is time to end their games and show them what you really think of them and the system they created.

The admins have patched the previous vulnerabilities and added new features to the platform. Now players can customize their display name! They also monitor the platform closely, reviewing reported content immediately.

Goal: To get the flag, reset ALL users' balance and score to 0 by exploiting the admin's privileges.

Key Learning Goals

This challenge introduces the following concepts:

- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF) and its defenses
- How XSS can bypass CSRF protections

Concepts from Lecture

What is XSS?

Cross-Site Scripting (XSS) allows an attacker to inject malicious JavaScript into a web page viewed by other users. When the victim's browser loads the page, the injected script executes with the same privileges as legitimate scripts.

There are two main types:

- **Reflected XSS** — payload is part of the request (e.g., URL parameter)
- **Stored XSS** — payload is saved on the server and served to all users who view the affected page

Example:

```
<script>alert('test')</script>
```

What is CSRF?

Cross-Site Request Forgery (CSRF) tricks a victim's browser into making an unwanted request to a site where they are authenticated. Typical CSRF defenses include tokens that verify the request originated from the legitimate site.